Orderly

Decisões técnicas, limitações conhecidas e possíveis evoluções

Decisões técnicas

Outbox transacional

Gravar o pedido no banco e publicar no RabbitMQ são duas operações em sistemas diferentes. Se a aplicação publicasse direto na requisição, uma falha entre as duas deixaria um pedido sem mensagem (nunca processado) ou uma mensagem sem pedido.

A solução foi o padrão **outbox**: na mesma transação em que o pedido é salvo, é gravado também um registro na tabela `order_outbox`. Ou tudo é salvo, ou nada. Um job agendado (`OrderOutboxRelay`) lê os registros pendentes e publica no RabbitMQ, esperando a confirmação do broker (*publisher confirms*) antes de marcá-los como publicados. Se o RabbitMQ estiver fora do ar, o pedido é criado normalmente e a mensagem é publicada quando ele voltar.

A busca usa `FOR UPDATE SKIP LOCKED`, o que permite rodar mais de uma instância do backend sem publicar o mesmo registro duas vezes.

Entrega "pelo menos uma vez" e consumidor idempotente

O outbox garante que a mensagem será publicada, mas não que será publicada **só uma vez**: se o broker confirmar e o banco falhar antes de marcar o registro, ele é publicado de novo. O RabbitMQ também pode reentregar mensagens. Por isso o consumidor foi feito para ignorar repetições:

- antes de processar, verifica se o pedido já está finalizado (`COMPLETED` ou `FAILED`) e, se estiver, ignora a mensagem;
- a coluna `version` (`@Version` do JPA) impede que duas execuções simultâneas finalizem o mesmo pedido: a segunda falha ao gravar, é tentada de novo, encontra o pedido finalizado e é ignorada.

O `orderId` funciona como chave de idempotência natural, então não foi necessário criar uma chave extra na mensagem.

Mensagem com apenas o identificador

A mensagem leva só o `orderId`. O consumidor lê o estado atual no banco, que é a fonte da verdade, e por isso uma mensagem atrasada ou repetida nunca traz dados desatualizados.

Duas transações curtas no processamento

O consumidor muda o pedido para `PROCESSING` em uma transação, processa fora de transação e grava o resultado em outra. Assim o status `PROCESSING` fica visível para o frontend durante o processamento, e nenhuma linha do banco fica travada durante a espera.

Falha de negócio x falha técnica

- **Falha de negócio** (pedido recusado): é um resultado válido. O pedido vai para `FAILED` com o motivo, sem novas tentativas, porque tentar de novo daria o mesmo resultado.
- **Falha técnica** (exceção): pode ser passageira. O Spring AMQP tenta de novo com intervalo crescente e, esgotadas as tentativas, o `OrderProcessingRecoverer` marca o pedido como `FAILED` e rejeita a mensagem, que vai para a DLQ para análise.

Total calculado pelo servidor

O total é sempre a soma de `quantidade × preço` de cada item, calculada na entidade `Order`. O frontend mostra um total estimado apenas como referência. O banco reforça a regra com um `CHECK` em `line_total`.

Paginação, ordenação e filtros no servidor

Como a paginação é feita no banco, a ordenação também precisa ser. Ordenar só a página atual no navegador daria um resultado errado. A ordenação aceita apenas campos conhecidos e usa o `id` como desempate, para que nenhum pedido se repita ou suma entre páginas. Os filtros de data consideram o dia inteiro no fuso configurado (`America/Sao_Paulo`).

Erros padronizados

Todos os erros seguem o ProblemDetail, com mensagens em português. Erros de validação trazem o caminho do campo (por exemplo `items[0].quantity`), que o formulário usa para marcar o campo certo.

Frontend com contrato da API e implementação trocável

As telas dependem só da classe abstrata `OrdersApi`. Isso permitiu construir e testar as telas com dados simulados antes do backend existir, e trocar para a API real mudando uma linha.

Proxy reverso no nginx

O frontend é servido pelo nginx, que também repassa `/api` e `/swagger-ui` para o backend. O navegador fala com uma única origem, o que evita CORS e esconde a porta do backend.

Limitações conhecidas e possíveis evoluções

Limitações

- **Paginação por offset:** cada página faz um `count(*)` para calcular o total. Com milhões de registros isso ficaria lento.
- **Processamento simulado:** a validação de negócio usa regras fixas (palavras-chave e valor máximo) apenas para demonstrar os cenários.
- **Mensagens na DLQ não são reprocessadas automaticamente:** ficam na fila para análise manual.

Possíveis evoluções

- Paginação por cursor (*keyset pagination*) para grandes volumes.
- Índices para ordenação por cliente e por valor.
- Atualização em tempo real, no lugar do polling.
- Tela ou endpoint para reprocessar mensagens da DLQ.
- Testes end-to-end do frontend (por exemplo, Playwright).